Edge Audio Intelligence System

Pipeline Architecture & Algorithm Reference

EE678 — Wavelets and Multiresolution Signal Processing

Pipeline Reference Document

March 2026

1 System Overview

A modular, multi-resolution edge audio pipeline for real-time speech processing. Six stages run in cascade; each stage adds keys to a shared dictionary that flows downstream.

V1 Cascade Pipeline. Each module appends keys to the shared data dictionary.

Signal Model. The m-th microphone receives:

$$x_m(t) = \sum_{s=1}^{S} h_{ms}(t) * s_s(t) \; + \; n_m(t) \tag{1}$$

where h_{ms} is the room impulse response (RIR) from source s to mic m, s_s is the clean source signal, and n_m is additive noise. RIRs are generated via the **Image Source Method** (pyroomacoustics).

Default Setup: 4-mic linear array, 5 cm spacing, room $6 \times 5 \times 3$ m, $f_s = 16$ kHz, seed 42.

2 Stage 1 — Sound Source Localization (SSL)

Estimates the direction-of-arrival (DOA) $\hat{\theta}$ of each active source from the multichannel input.

GCC-PHAT (default)

Idea. Cross-correlate a microphone pair with phase-transform weighting to find the time-delay of arrival (TDOA), then convert to DOA.

Algorithm.

1. Select the widest-baseline mic pair (best angular resolution).
2. Cross-power spectrum with PHAT weighting:

$$R_{xy}(f) = \frac{X(f)\,Y^*(f)}{|X(f)\,Y^*(f)|} \tag{2}$$

3. Inverse FFT to get the generalized cross-correlation:

$$\mathrm{gcc}(\tau) = \mathrm{IFFT}\{R_{xy}(f)\} \tag{3}$$

4. Sub-sample peak via 3-point parabolic interpolation $\Rightarrow$ $\hat{\tau}$ samples.
5. Convert TDOA to DOA:

$$\hat{\theta} = \arccos\left(\frac{\hat{\tau}\,c}{d\,f_s}\right), \qquad c = 343 \text{ m/s} \tag{4}$$

Parameters: `n_fft` $= 1024$. **Complexity:** $\mathcal{O}(N \log N)$ — three FFTs.

SRP-PHAT

Idea. Steer a spatial grid of candidate directions; sum GCC-PHAT values at the corresponding TDOA for each pair. The grid maximum is the DOA.

$$P_{\text{SRP}}(\theta) = \sum_{\text{pairs}} \sum_{f} \text{GCC-PHAT}\big(\tau_{ij}(\theta),\, f\big), \qquad \tau_{ij}(\theta) = \frac{(\mathbf{p}_i - \mathbf{p}_j) \cdot \mathbf{u}(\theta)}{c} \tag{5}$$

Parameters: `n_fft` = 1024, grid resolution = 360 (1°). **Complexity:** $\mathcal{O}(G \cdot P \cdot F)$ where G=grid, P=pairs, F=freq bins.

MUSIC

Idea. Eigendecompose the spatial covariance; scan for directions whose steering vector is orthogonal to the noise subspace.

$$\mathbf{R}_{xx}(f) = \frac{1}{T} \sum_{t} \mathbf{X}(f,t)\, \mathbf{X}^H(f,t) = \mathbf{U}_s \mathbf{\Lambda}_s \mathbf{U}_s^H + \mathbf{U}_n \mathbf{\Lambda}_n \mathbf{U}_n^H \tag{6}$$

$$P_{\text{MUSIC}}(\theta) = \frac{1}{\mathbf{a}^H(\theta)\, \mathbf{U}_n \mathbf{U}_n^H\, \mathbf{a}(\theta)} \tag{7}$$

Parameters: `n_fft` = 1024, `n_sources` = 1, grid = 360. **Complexity:** $\mathcal{O}(M^3 F + G\, M^2 F)$.

3 Stage 2 — Beamforming

Combines multichannel signals into a single enhanced channel steered towards $\hat{\theta}$.

Delay-and-Sum (default)

Per-mic steering delay from the estimated DOA:

$$\tau_m = \frac{(\mathbf{p}_m - \mathbf{p}_{\text{center}}) \cdot \mathbf{u}(\hat{\theta})}{c} \tag{8}$$

Fractional delay applied in the frequency domain:

$$X'_m(f) = X_m(f)\, \exp\big(-\, j\, 2\pi f\, \tau_m / N\big) \tag{9}$$

Output is the average: $y(t) = \dfrac{1}{M} \displaystyle\sum_{m=1}^{M} x_m(t - \tau_m)$.

Array gain in spatially white noise: $G = M$ ($= 4$ for the default array).
Parameters: `use_fractional_delay` = `True`. **Complexity:** $\mathcal{O}(M\, N \log N)$.

MVDR (Minimum Variance Distortionless Response)

Minimizes output power subject to unity gain in the look direction.

$$\mathbf{R}(f) = \frac{1}{T}\sum_t \mathbf{X}(f,t)\,\mathbf{X}^H(f,t) + \lambda\,\frac{\mathrm{tr}(\mathbf{R})}{M}\,\mathbf{I} \qquad \text{(diagonal loading)} \tag{10}$$

$$\mathbf{w}(f) = \frac{\mathbf{R}^{-1}(f)\,\mathbf{a}(f)}{\mathbf{a}^H(f)\,\mathbf{R}^{-1}(f)\,\mathbf{a}(f)} \tag{11}$$

$$Y(f,t) = \mathbf{w}^H(f)\,\mathbf{X}(f,t) \tag{12}$$

Parameters: `n_fft` = 512, `hop` = 256, $\lambda = 0.01$. **Complexity:** $\mathcal{O}(F\,M^3)$ per frame (matrix inversion).

4 Stage 3 — Speech Enhancement

Removes residual noise from the beamformed signal.

Spectral Subtraction (default)

1. STFT $\Rightarrow$ $X(f,t)$.
2. Estimate noise power from first K frames: $\hat{N}(f) = \frac{1}{K}\sum_{t=1}^{K} |X(f,t)|^2$.
3. Subtract with oversubtraction α and spectral floor β:

$$|\hat{S}(f,t)|^2 = \max\Big(|X(f,t)|^2 - \alpha\,|\hat{N}(f)|^2,\;\; \beta\,|X(f,t)|^2\Big) \tag{13}$$

4. Reconstruct with noisy phase: $\hat{S}(f,t) = |\hat{S}(f,t)|\,e^{j\angle X(f,t)}$.
5. ISTFT $\Rightarrow$ $\hat{s}(t)$.

Parameters: `n_fft` = 512, `hop` = 256, $\alpha = 2.0$, $\beta = 0.01$, noise frames = 10, noise update rate = 0.98.

Wavelet Enhancement (wavelet integration)

Key course contribution. Exploits the fact that speech energy concentrates in low sub-bands while broadband noise is uniformly distributed.

1. **J-level DWT** (CDF 5/3, `bior2.2`, $J = 3$):

$$\text{signal} \xrightarrow{\text{DWT}} [cA_3,\; cD_3,\; cD_2,\; cD_1]$$

2. **Noise estimation** via Median Absolute Deviation on finest detail:

$$\hat{\sigma} = \frac{\mathrm{median}(|cD_1|)}{0.6745} \tag{14}$$

3. **Level-dependent soft thresholding:**

$$T_j = \hat{\sigma}\,\sqrt{2\ln N}\;\cdot\;\mathrm{scale}_j, \qquad \widetilde{cD}_j = \mathrm{sign}(cD_j)\,\max\big(|cD_j| - T_j,\;0\big) \tag{15}$$

   Coarser levels get lower thresholds (more speech there).
4. **Inverse DWT:** $\hat{s}(t) = \mathrm{IDWT}\big[cA_3,\; \widetilde{cD}_3,\; \widetilde{cD}_2,\; \widetilde{cD}_1\big]$

Sub-band frequency ranges ($f_s = 16\,\mathrm{kHz}$, $J = 3$):

Coefficient	Freq. Range	Content
cA_3	0–1 kHz	Speech fundamentals
cD_3	1–2 kHz	Speech formants
cD_2	2–4 kHz	Consonants / noise
cD_1	4–8 kHz	Mostly noise

Parameters: wavelet=`bior2.2`, levels= 3, mode=`soft`, threshold_scale= 1.0.
Complexity: $\mathcal{O}(N)$ total — linear, faster than STFT's $\mathcal{O}(N \log N)$.

5 Stage 4 — Automatic Speech Recognition (ASR)

Whisper Offline ASR

OpenAI Whisper encoder–decoder transformer. Processes the enhanced audio in batch mode.

Model	Params	Layers	Use Case
tiny	39 M	4	Quick tests
base	**74 M**	**6**	**Default (speed/accuracy)**
small	244 M	12	Better accuracy
medium	769 M	24	High accuracy
large	1.55 B	32	Best accuracy

Input: 16 kHz mono audio $\Rightarrow$ 80-channel log-mel spectrogram.
Output: text, word-level timestamps.
Parameters: `model_size=base`, `language=en`, `device=cpu`.

6 Wavelet Subsystem (Course Integration)

Four integration points ensure substantive wavelet usage for EE678:

ψ Wavelet Energy VAD

Ultra-low-power voice activity detector (< 5 mW target). Gates the entire pipeline.

1. Per-frame DWT $\Rightarrow$ sub-band energies $E(cA_3)$, $E(cD_3)$, $E(cD_2)$, $E(cD_1)$.
2. Energy ratio:
$$r = \frac{E(cA_3) + E(cD_3)}{E(cD_1) + \varepsilon} \tag{16}$$
Speech $\Rightarrow$ high r (energy in low bands); noise only $\Rightarrow$ low r.
3. VAD $= \mathbf{1}[r > T]$ with minimum-duration and hangover smoothing.

Parameters: frame=512 (32 ms), hop=256 (16 ms), $T = 3.0$, hangover=5 frames.
Complexity: $\sim 4N$ per DWT level + energy sums. Far cheaper than STFT-based VAD.

ψ Wavelet-Initialized CNN Kernels

For the multi-stream CNN SSL (V2 scope): initialize parallel convolution kernels with Daubechies filter coefficients.

Kernel Size	Wavelet	Filter Taps
3	db2	4
5	db3	6
7	db4	8

Each kernel gets lowpass + highpass variants; small random perturbation breaks symmetry. Benefit: faster convergence and better-conditioned multi-scale features.

Interpretability Analysis

At each pipeline stage, compute the wavelet scalogram and sub-band energies to visualize how information flows and transforms. Demonstrates that the pipeline is not a black box — each stage has an explainable multi-resolution structure.

7 Evaluation Framework

7.1 Metrics per Stage

Stage	Metric	Formula / Range	Goal
SSL	Angular Error	$\min(\lvert\Delta\theta\rvert,\ 360-\lvert\Delta\theta\rvert)$	$\downarrow$
SSL	RMSAE	$\sqrt{\frac{1}{N}\sum_i e_i^2}$	$\downarrow$
Beamforming	SI-SDR	$10\log_{10}\frac{\lVert\mathbf{s}_{\text{target}}\rVert^2}{\lVert\mathbf{e}\rVert^2}$ (dB)	$\uparrow$
Enhancement	PESQ	ITU-T P.862, range $[-0.5,\ 4.5]$	$\uparrow$
Enhancement	STOI	Short-Time Intelligibility, range $[0,\ 1]$	$\uparrow$
Enhancement	SI-SDR	(same as above)	$\uparrow$
ASR	WER	$\frac{S+D+I}{N_{\text{ref}}}$ (Levenshtein)	$\downarrow$
System	RTF	processing time / audio duration	<1

SI-SDR detail: $\mathbf{s}_{\text{target}} = \frac{\langle\hat{s}, s\rangle}{\lVert s\rVert^2}\, s, \quad \mathbf{e} = \hat{s} - \mathbf{s}_{\text{target}}.$

7.2 SNR × RT60 Evaluation Grid

Every module comparison runs on this standardized grid:

	SNR = 30 dB	**SNR = 15 dB**	**SNR = 5 dB**
RT60 = 0.0 s	LL (easy)		HL (noisy)
RT60 = 0.3 s		•	
RT60 = 0.6 s		•	
RT60 = 0.9 s		•	
RT60 = 1.2 s		•	
RT60 = 1.5 s	LH (reverb)		HH (hard)

Corner cases (LL, HL, LH, HH) are used for quick checks; the full $3 \times 6 = 18$ grid for thorough evaluation.

8 Testbench & Simulation

- **RIR generation:** Image Source Method via `pyroomacoustics.ShoeBox` with RT60-based absorption.
- **Mic presets:** Linear (4 mic, 5 cm), circular (4 mic, 4.25 cm radius), phone (2 mic, 7 cm), smart speaker.
- **Noise types:** white, babble, from file. Mixed at target SNR.
- **Sources:** speech (from LibriSpeech), sine, chirp, white noise.
- **Reproducibility:** seeded RNG per scene; full config logged as JSON.

9 Pipeline Orchestration

- **CascadePipeline:** Sequential module chain. Calls `module.process(data)` for each stage. Records per-module timing.

- **ExperimentRunner:** Generates scenes → runs pipeline → evaluates metrics → saves JSON → appends to `RESEARCH_LOG.md`.
- **Data contract:** Every module receives and returns a `Dict[str, Any]`. Keys are standardized (Section 1 diagram). Modules only *add* keys; never remove.

10 Default Pipeline Configuration

Parameter	Value	Source
Sample rate	16 kHz	`default.yaml`
Room	$6 \times 5 \times 3$ m	`default.yaml`
RT60	0.3 s	`default.yaml`
SNR	15 dB	`default.yaml`
Mic array	4-mic linear, 5 cm	`default.yaml`
SSL	GCC-PHAT, `n_fft`=1024	`default.yaml`
Beamformer	Delay-and-Sum, fractional delay	`default.yaml`
Enhancement	Spectral Subtraction, $\alpha = 2$, $\beta = 0.01$	`default.yaml`
ASR	Whisper base, English	`default.yaml`
Evaluation	Corner cases or full grid	`default.yaml`
Seed	42	`default.yaml`

11 Implementation Status

Component	V1 (Current)	V2 (Planned)
SSL: GCC-PHAT, SRP-PHAT, MUSIC	✓	—
SSL: Multi-stream CNN	—	planned
Beamforming: DS, MVDR	✓	—
Enhancement: Spectral Sub., Wavelet	✓	—
Wavelet VAD, Init, Analysis	✓	—
Separation: Conv-TasNet	—	planned
ASR: Whisper Offline	✓	—
ASR: Emformer Streaming	—	planned
Diarization: pyannote	—	planned
Feedback loop (Level 2)	—	planned